

Milestone 3 - Project Outline

Group 6

SQL Queries

1 Top 50 Songs for a Certain Country

```
SELECT
  MIN(s.song_name) AS song_name,
  STRING_AGG(DISTINCT a.artist_name, ', ') AS artists,
  MIN(ce.chart_position) AS best_position,
  MAX(ce.chart_date) AS most_recent_date
FROM chart_entry ce
  JOIN song s ON ce.song_id = s.song_id
  JOIN song_artist sa ON s.song_id = sa.song_id
  JOIN artist a ON sa.artist_id = a.artist_id
WHERE ce.country_code = 'USA'
  AND ce.chart_date >= '2024-11-01'
GROUP BY s.song_id
ORDER BY best_position ASC, most_recent_date DESC
LIMIT 50;
```

Description: Finds the top 50 songs for a certain country in the most recent month (defaults to US, but can change for other countries). It is ranked by the best chart position reached, then the most recent date.

2 Top 50 Songs Globally (based on global reach)

```
SELECT
  MIN(s.song_name) AS song_name,
  STRING_AGG(DISTINCT a.artist_name, ', ') AS artists,
  COUNT(DISTINCT ce.country_code) AS countries_charted,
  MIN(ce.chart_position) AS best_position,
  MAX(ce.chart_date) AS latest_date
FROM chart_entry ce
  JOIN song s ON ce.song_id = s.song_id
  JOIN song_artist sa ON s.song_id = sa.song_id
  JOIN artist a ON sa.artist_id = a.artist_id
GROUP BY s.song_id
ORDER BY countries_charted DESC, best_position ASC
LIMIT 50;
```

Description: Finds the top 50 songs with the most reach globally. It ranks songs by the number of countries that they have charted (and if there is a tie, it favors the best chart position achieved).

3 Top Artists Globally

```
SELECT
  a.artist_id,
  a.artist_name,
  STRING_AGG(DISTINCT a.artist_genre, ', ') AS artist_genres,
  COUNT(DISTINCT s.song_id) AS total_charting_songs,
  ROUND(AVG(ce.chart_position), 2) AS avg_chart_position,
  COUNT(DISTINCT ce.country_code) AS countries_charted,
  MIN(ce.chart_position) AS best_position
FROM artist a
JOIN song_artist sa ON a.artist_id = sa.artist_id
JOIN song s ON sa.song_id = s.song_id
JOIN chart_entry ce ON s.song_id = ce.song_id
WHERE ce.chart_date = (SELECT MAX(chart_date) FROM
chart_entry)
GROUP BY a.artist_id, a.artist_name
ORDER BY avg_chart_position ASC, total_charting_songs DESC
LIMIT 50;
```

Description: Finds the top artists in the world based on their average chart performance across all countries for the most recent chart date. It ranks them by average chart position, and ties are broken by total number of charting songs.

4 Top Genres Globally

```
SELECT
  a.artist_genre,
  COUNT(DISTINCT s.song_id) AS songs_in_chart,
  ROUND(AVG(ce.chart_position), 2) AS avg_position
FROM chart_entry ce
JOIN song s ON ce.song_id = s.song_id
JOIN song_artist sa ON s.song_id = sa.song_id
JOIN artist a ON sa.artist_id = a.artist_id
WHERE ce.chart_date = (SELECT MAX(chart_date) FROM
```

```

chart_entry)
    AND a.artist_genre IS NOT NULL
GROUP BY a.artist_genre
ORDER BY songs_in_chart DESC
LIMIT 50;

```

Description: Finds the most popular genres globally based on the most recent chart date. It ranks genres by number of songs charting for each genre

5 Top Songs by a Certain Artist

```

SELECT
    MIN(s.song_name) AS song_name,
    MIN(ce.chart_position) AS best_position,
    COUNT(DISTINCT ce.country_code) AS countries_charted,
    MAX(ce.chart_date) AS most_recent_date
FROM artist a
    JOIN song_artist sa ON a.artist_id = sa.artist_id
    JOIN song s ON sa.song_id = s.song_id
    JOIN chart_entry ce ON s.song_id = ce.song_id
WHERE a.artist_id = '06HL4z0CvFAXyc27GXpf02'
GROUP BY s.song_id
ORDER BY best_position ASC, most_recent_date DESC
LIMIT 50;

```

Description: Finds the top 50 songs of a certain artist (in this example, it's Taylor Swift). It's ranked by best chart position across all of time and all countries.

*** *change a.artist_id for desired artist*

6 Song Duration by Genre and Country

```

SELECT
    c.country_name,
    a.artist_genre,
    ROUND(AVG(s.duration) / 60000.0, 2) AS avg_duration_minutes,
    COUNT(DISTINCT s.song_id) AS song_count
FROM chart_entry ce
    JOIN song s ON ce.song_id = s.song_id
    JOIN song_artist sa ON s.song_id = sa.song_id

```

```

JOIN artist a ON sa.artist_id = a.artist_id
JOIN country c ON ce.country_code = c.country_code
WHERE ce.chart_position <= 50
      AND ce.chart_date = (SELECT MAX(chart_date) FROM chart_entry)
      AND a.artist_genre IS NOT NULL
      AND s.duration IS NOT NULL
GROUP BY c.country_name, a.artist_genre
HAVING COUNT(DISTINCT s.song_id) >= 3
ORDER BY avg_duration_minutes DESC, c.country_name;

```

Description: Examines the average duration (in minutes) of a song for different genres and different countries. This can be used to determine regional trends of song duration.

7 Changes in Genre Over Time

```

WITH monthly_stats AS (
  SELECT
    DATE_TRUNC('month', ce.chart_date) AS month,
    a.artist_genre,
    ROUND(AVG(ce.chart_position), 2) AS avg_position,
    COUNT(DISTINCT s.song_id) AS songs_charted
  FROM chart_entry ce
  JOIN song s ON ce.song_id = s.song_id
  JOIN song_artist sa ON s.song_id = sa.song_id
  JOIN artist a ON sa.artist_id = a.artist_id
  WHERE ce.chart_date >= '2023-11-01'
  GROUP BY DATE_TRUNC('month', ce.chart_date), a.artist_genre
  HAVING COUNT(DISTINCT s.song_id) >= 5
),
latest_vs_oldest AS (
  SELECT
    artist_genre,
    MAX(CASE WHEN month = '2024-11-01' THEN avg_position
  END) AS nov_2024_position,
    MAX(CASE WHEN month = '2023-11-01' THEN avg_position
  END) AS nov_2023_position,
    MAX(CASE WHEN month = '2024-11-01' THEN songs_charted
  END) AS nov_2024_songs,
    MAX(CASE WHEN month = '2023-11-01' THEN songs_charted
  END) AS nov_2023_songs

```

```

FROM monthly_stats
GROUP BY artist_genre
)
SELECT
    artist_genre,
    nov_2023_position,
    nov_2024_position,
    ROUND(nov_2023_position - nov_2024_position, 2) AS
position_improvement,
    nov_2023_songs,
    nov_2024_songs,
    nov_2024_songs - nov_2023_songs AS song_growth
FROM latest_vs_oldest
WHERE nov_2023_position IS NOT NULL AND nov_2024_position IS
NOT NULL
ORDER BY position_improvement DESC;

```

Description: This query can be used to determine genre trends over time between Nov 2023 and Nov 2024. It examines how the popularity of the chosen genre has changed over time, specifically looking at the average chart position and number of songs on the chart for certain genres.

8 Compare a Certain Song Across Countries

```

SELECT
    c.country_name,
    ce.chart_position,
    ce.chart_date
FROM chart_entry ce
    JOIN country c ON ce.country_code = c.country_code
WHERE ce.song_id = '1BxfuPKGuaTgP7aM0Bbdwr'
ORDER BY ce.chart_position ASC, ce.chart_date DESC;

```

Description: Finds the chart performance of a specific song across all countries and dates (where the song charted and when) ordered by best chart position, then chart date. The example is for “Cruel Summer” by Taylor Swift, but can be changed for other songs.

9 Artists that Dominate a Certain Country

WITH ranked AS (

```

SELECT
    c.country_name,
    MIN(a.artist_name) AS artist_name,
    COUNT(DISTINCT s.song_id) AS songs_in_top_50,
    ROUND(AVG(ce.chart_position), 2) AS avg_position,
    MIN(ce.chart_position) AS best_position,
    ROW_NUMBER() OVER (PARTITION BY c.country_name
ORDER BY COUNT(DISTINCT s.song_id) DESC,
AVG(ce.chart_position) ASC) as rank
FROM chart_entry ce
JOIN song s ON ce.song_id = s.song_id
JOIN song_artist sa ON s.song_id = sa.song_id
JOIN artist a ON sa.artist_id = a.artist_id
JOIN country c ON ce.country_code = c.country_code
WHERE ce.chart_date = (SELECT MAX(chart_date) FROM
chart_entry)
    AND ce.chart_position <= 50
    GROUP BY c.country_name, a.artist_id
    HAVING COUNT(DISTINCT s.song_id) >= 2
)
SELECT country_name, artist_name, songs_in_top_50, avg_position,
best_position
FROM ranked
WHERE rank = 1
ORDER BY songs_in_top_50 DESC, avg_position ASC;

```

Description: Finds the dominant artist for the country's market for the most recent date. It is ordered by the number of top songs in the top 50, then their average position.

10 New Songs in the Charts

```

SELECT
    c.country_name,
    MIN(s.song_name) AS song_name,
    STRING_AGG(DISTINCT a.artist_name, ', ') AS artists,
    ce.chart_position,
    ce.chart_date AS entry_date
FROM chart_entry ce
JOIN song s ON ce.song_id = s.song_id
JOIN song_artist sa ON s.song_id = sa.song_id

```

```

        JOIN artist a ON sa.artist_id = a.artist_id
        JOIN country c ON ce.country_code = c.country_code
WHERE ce.chart_date = (SELECT MAX(chart_date) FROM
chart_entry)
    AND ce.chart_position <= 50
    AND NOT EXISTS (
        SELECT 1
        FROM chart_entry ce2
        WHERE ce2.song_id = ce.song_id
            AND ce2.country_code = ce.country_code
            AND ce2.chart_date < ce.chart_date
    )
GROUP BY c.country_name, s.song_id, ce.chart_position,
ce.chart_date
ORDER BY ce.chart_position ASC, c.country_name
LIMIT 50;

```

Description: Finds songs that are new to the chart by showing songs that never charted in that country before. It shows songs that have just become very popular and are fresh to the charts.

Info to Access Database

Guest Information:

Host: cis550group6.cleekk82kxqm.us-east-1.rds.amazonaws.com

Port: 5432

Database: music_group6

Username: guest_user

Password: guest_group6

Instructions:

- 1 Open DataGrip
- 2 Connect to the data using the credentials provided
- 3 Now you will have read-only access to the database

Master Information (if needed):

Master UserName: fall25group6

Master Password: MSeattle7833Key

Host: cis550group6.cleekk82kxqm.us-east-1.rds.amazonaws.com

Port: 5432